
Manual Técnico 09

Mensageria e Desacoplamento com RabbitMQ

Infraestruturas de Computação na Nuvem | Aula 09

Como usar este manual

O objetivo central desta aula cabe numa frase: **ver o produtor continuar a funcionar quando o consumidor desaparece**. Tudo o resto é preparação para chegar a esse momento.

Duas vias possíveis. Este manual segue a via principal, com o cluster Kubernetes. Se o cluster não estiver disponível, a secção «Via alternativa com Docker» no fim permite fazer as partes essenciais dentro da VM `icn-lab`.

Antes de começar

- ☐ Cluster Kubernetes com dois nós `Ready` (ou VM com Docker, para a via alternativa)
- ☐ *Taint* do control plane removido – necessário para o exercício de anti-affinity da Parte 6:

```
$ kubectl taint nodes --all node-role.kubernetes.io/control-plane- 2>/dev/null
```

- ☐ Dois ou três terminais SSH abertos
- ☐ Acesso ao Proxmox para o teste final

1 Implantar o RabbitMQ

1.1 Aplicar o manifesto

```
$ nano rabbitmq.yaml
```

Copie o conteúdo do Guião 09.

```
$ kubectl apply -f rabbitmq.yaml
$ kubectl get pods -l app=rabbitmq --watch
```

Atenção

O arranque demora **30 a 60 segundos**. Espere pelo estado `Running` com 1/1 na coluna `READY`. Se avançar antes disso, as ligações vão falhar. Interrompa o `-watch` com `Ctrl+C` quando estiver pronto.

1.2 Aceder à interface de gestão

Abra no browser:

```
http://<IP-de-um-no>:31672
```

Credenciais: as que definiu no Secret (`aluno / aulaicn2026`).

Confirme antes de avançar

Entrou na interface e vê os separadores *Overview*, *Connections*, *Exchanges* e *Queues*. Vai usá-los ao longo da aula para ver o que se passa por dentro.

1.3 Confirmar o DNS interno

```
$ kubectl run dns-teste --rm -it --image=busybox:1.36 --restart=Never -- \
  nslookup rabbitmq
```

Saída esperada

```
Name: rabbitmq
Address 1: 10.106.55.12 rabbitmq.default.svc.cluster.local
```

Os Pods poderão alcançar o broker pelo nome `rabbitmq`, sem saber o IP.

2 Work queue

2.1 Preparar os scripts

```
$ nano worker.py
```

Copie o `worker.py` do Guião 09. Dois detalhes que valem atenção:

- `basic_qos(prefetch_count=1)` – cada worker só recebe uma tarefa de cada vez, o que distribui o trabalho de forma justa.
- `basic_ack` no fim do `callback` – confirma o processamento. Sem esta confirmação, o broker volta a entregar a mensagem a outro worker.

2.2 Injetar o código no cluster

```
$ kubectl create configmap worker-code --from-file=worker.py
$ kubectl get configmap worker-code
```

Usamos um ConfigMap para evitar construir e distribuir uma imagem própria. Em produção construir-se-ia uma imagem; aqui, para uma aula, isto é mais rápido.

2.3 Implantar os workers

```
$ nano worker.yaml
$ kubectl apply -f worker.yaml
$ kubectl get pods -l app=worker
```

Atenção

Cada Pod faz `pip install pika` ao arrancar, o que demora e exige acesso à Internet a partir dos nós. Espere que fiquem **Running**. Se ficarem em **Error** ou **CrashLoopBackOff**, veja os logs.

2.4 Acompanhar os logs

Terminal 1:

```
$ kubectl logs -f -l app=worker --prefix --max-log-requests=10
```

Saída esperada

```
[pod/worker-6c9f8d4b7-x2kp9/worker] [worker-6c9f8d4b7-x2kp9] A espera de tarefas.
[pod/worker-6c9f8d4b7-mz4tq/worker] [worker-6c9f8d4b7-mz4tq] A espera de tarefas.
```

2.5 Enviar tarefas

A forma mais simples é pela interface web:

1. Separador **Queues** → clique em **tarefas**
2. Secção **Publish message**
3. Em *Payload*, escreva **tarefa 1**
4. **Publish message**
5. Repita com **tarefa 2**, **tarefa 3**, **tarefa 4**...

Confirme antes de avançar

No Terminal 1, as tarefas aparecem distribuídas por workers *diferentes*. Cada tarefa é processada por **um** worker apenas. Registe a distribuição.

2.6 Escalar os consumidores

```
$ kubectl scale deployment worker --replicas=4
$ kubectl get pods -l app=worker
```

Envie mais tarefas seguidas e compare o tempo total de processamento. Cada tarefa demora 2 segundos (`time.sleep(2)`): com 2 workers, 8 tarefas levam cerca de 8 segundos; com 4 workers, cerca de 4.

3 Publish/subscribe

Adapte o `worker.yaml` para os subscribers (mude o nome do Deployment, o ConfigMap e o script executado), usando o `subscribe_event.py` do guião.

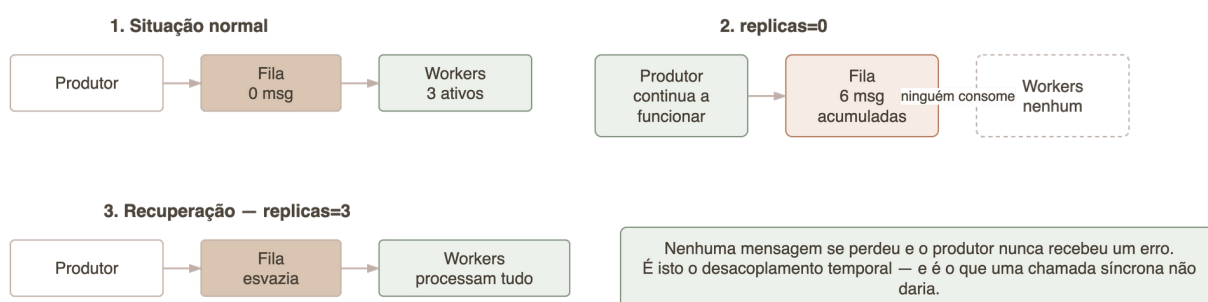
Publique um evento pela interface web, agora no exchange **eventos** (separador **Exchanges**).

Confirme antes de avançar

Todos os subscribers receberam a *mesma* mensagem. Compare com a work queue, onde cada mensagem foi para *um* consumidor só. A diferença está no tipo de exchange: **fanout** difunde, a fila directa distribui.

No separador **Queues** confirme que cada subscriber tem a sua própria fila temporária ligada ao exchange.

4 O exercício central: desacoplamento



O produtor nunca falha, mesmo sem nenhum consumidor ativo.

4.1 Desligar todos os consumidores

```
$ kubectl scale deployment worker --replicas=0
$ kubectl get pods -l app=worker
```

Saída esperada

```
No resources found in default namespace.
```

4.2 Continuar a produzir

Publique 5 ou 6 mensagens pela interface web, exatamente como antes.

Confirme antes de avançar

O produtor funcionou normalmente. Nenhum erro, nenhuma exceção, nenhuma espera — apesar de não existir um único consumidor. É isto o desacoplamento temporal.

4.3 Ver as mensagens acumuladas

Na interface web, separador **Queues**, a coluna *Ready* da fila **tarefas** mostra as mensagens à espera. Ou por linha de comandos:

```
$ kubectl exec deployment/rabbitmq -- rabbitmqctl list_queues name messages
```

Saída esperada

```
Listing queues ...
name messages
tarefas 6
```

Registre o número: _____

4.4 Recuperar

```
$ kubectl scale deployment worker --replicas=3
$ kubectl logs -f -l app=worker --prefix --max-log-requests=10
```

Confirme antes de avançar

Os workers arrancaram e processaram **todas** as mensagens acumuladas, sem perda. O sistema recuperou sozinho de uma indisponibilidade total do lado consumidor.

Pense no que aconteceria numa chamada síncrona (REST direto): o produtor teria recebido erro de ligação em cada tentativa, e o trabalho ter-se-ia perdido — a menos que implementasse ele próprio filas, retries e persistência. A fila resolve isso de origem.

5 Durabilidade

```
$ kubectl scale deployment worker --replicas=0
```

Publique 3 mensagens. Depois reinicie o broker:

```
$ kubectl rollout restart deployment rabbitmq
$ kubectl get pods -l app=rabbitmq --watch
```

Quando voltar a **Running**, verifique:

```
$ kubectl exec deployment/rabbitmq -- rabbitmqctl list_queues name messages
```

Atenção

As mensagens desapareceram – e isso é esperado. Este Deployment não tem armazenamento persistente: quando o Pod é recriado, começa de zero. As mensagens estavam marcadas como duráveis, mas a durabilidade escreve em disco, e o disco deste Pod é efêmero. Para as preservar seria preciso um **StatefulSet** com um **PersistentVolume**. Esta é a resposta a uma das questões de verificação.

6 Redundância entre nós

6.1 Ver a distribuição

```
$ kubectl scale deployment worker --replicas=4
$ kubectl get pods -l app=worker -o wide
```

6.2 Forçar distribuição por nós diferentes

Acrescente o bloco `affinity` do Guião 09 ao `worker.yaml` e reaplique:

```
$ kubectl apply -f worker.yaml
$ kubectl get pods -l app=worker -o wide
```

Confirme antes de avançar

Os Pods estão agora repartidos pelos dois nós, em vez de concentrados num só. Se um nó cair, sobrevivem os do outro.

6.3 Testar a falha de nó

Na shell do node Proxmox:

```
$ qm stop <vmid-w1>
```

Publique mais mensagens e confirme que continuam a ser processadas pelos workers do nó sobrevivente.

```
$ qm start <vmid-w1>
```

7 Limpeza

```
$ kubectl delete -f worker.yaml
$ kubectl delete configmap worker-code
$ kubectl delete -f rabbitmq.yaml
$ kubectl get all
```

8 Via alternativa com Docker

Se o cluster não estiver disponível, na VM `icn-lab`:

```
$ docker run -d --name rabbitmq \
  -p 5672:5672 -p 15672:15672 \
  -e RABBITMQ_DEFAULT_USER=aluno \
  -e RABBITMQ_DEFAULT_PASS=aulaicn2026 \
  rabbitmq:3-management
$ docker ps
```

Interface em `http://<IP-da-VM>:15672`. Instale o cliente e corra os scripts localmente:

```
$ sudo apt install -y python3-pip
$ pip3 install pika
$ export RABBIT_HOST=localhost
$ python3 worker.py # num terminal
$ python3 send_task.py "tarefa 1" # noutro
```

O exercício central (secção 4) funciona igual: pare os workers com **Ctrl+C**, continue a enviar, e observe a fila a crescer.

9 Problemas frequentes

Os Pods do worker ficam em `CrashLoopBackOff`

Veja a causa:

```
$ kubectl logs -l app=worker --tail=30
```

As causas mais frequentes: o `pip install` falhou (sem Internet nos nós), ou o broker ainda não estava pronto quando o worker tentou ligar-se. No segundo caso, os Pods recuperam sozinhos ao fim de algumas tentativas.

`pika.exceptions.AMQPConnectionError`

O broker não está acessível. Confirme:

```
$ kubectl get pods -l app=rabbitmq
$ kubectl get svc rabbitmq
```

Confirme também que a variável `RABBIT_HOST` está definida como `rabbitmq`.

`ACCESS_REFUSED - Login was refused`

Utilizador ou password errados. Têm de coincidir com o Secret. Verifique o que o broker tem:

```
$ kubectl exec deployment/rabbitmq -- rabbitmqctl list_users
```

A interface web não abre em `:31672`

Confirme que o Service tem a NodePort correta e experimente o IP do outro nó:

```
$ kubectl get svc rabbitmq
```

As tarefas vão todas para o mesmo worker

Falta o `basic_qos(prefetch_count=1)`, ou as tarefas foram enviadas antes de o segundo worker estar pronto. Confirme que ambos aparecem no separador *Connections* da interface web.

`kubectl logs -l app=worker` só mostra um Pod

Acrescente `-max-log-requests=10` e `-prefix`, para ver todos e saber qual é qual.

As mensagens ficam presas em `Unacked`

Um worker recebeu a mensagem mas morreu antes do `ACK`. O broker volta a entregá-la quando a ligação cai. Se ficarem presas, reinicie os workers.

10 Referência rápida

Comando	Para que serve
<code>kubectl create configmap N --from-file=f</code>	Injetar ficheiro no cluster
<code>kubectl logs -f -l app=X --prefix</code>	Seguir logs de vários Pods
<code>kubectl scale deploy X --replicas=0</code>	Desligar todos os consumidores
<code>kubectl rollout restart deploy X</code>	Recriar os Pods
<code>kubectl exec deploy/rabbitmq - CMD</code>	Executar comando no broker
<code>rabbitmqctl list_queues name messages</code>	Mensagens por fila
<code>rabbitmqctl list_users</code>	Utilizadores definidos
<code>rabbitmqctl list_connections</code>	Ligações ativas

11 Checklist final

- ☐ O RabbitMQ está **Running** e acedo à interface de gestão
- ☐ Confirmei a resolução do nome `rabbitmq` por DNS interno
- ☐ Os workers arrancaram e vejo os logs de todos
- ☐ Registei a distribuição de tarefas pelos workers (work queue)
- ☐ Testei o publish/subscribe e confirmei que todos recebem a mesma mensagem
- ☐ **Reduzi os workers a zero e continuei a publicar sem erros**
- ☐ Registei o número de mensagens acumuladas na fila
- ☐ Voltei a escalar e as mensagens foram todas processadas
- ☐ Testei a durabilidade e sei explicar por que as mensagens se perderam
- ☐ Apliquei anti-affinity e confirmei a distribuição pelos nós
- ☐ Testei a falha de um nó com o sistema em funcionamento
- ☐ Limpei os recursos criados